RAJALAKSHMI ENGINEERING COLLEGE
RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23334 FUNDAMENTALS OF DATA SCIENCE

Laboratory Record Note Book

Name :	N VAIBHAVE
Year / Branch / Section :	2ND YEAR CSE G
University Register No. :	2116250701802
College Roll No. :	250701802
Semester :	3RD SEMESTER

Academic Year : 2026 - 2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CS23334 FUNDAMENTALS OF DATA SCIENCE

NAME	**VAIBHAVE N**
ROLL NO	**250701802**
YEAR	**2ND**
SEC	**G**

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: VAIBHAVE N

Academic Year: 2026-2027 Semester: 3rd Branch: CSE

Register No. 2116250701802

Certified that this is the bonafide record of work done by the above student in

the CS23334 FUNDAMENTALS OF DATA SCIENCE Laboratory

during the academic year 2025 – 2026.

Signature of Faculty in-charge

Submitted for the Practical Examination held on ……

Internal Examiner External Examiner

INDEX

SlNO	Experiment	Date	GitHub
1	**WEEK - 1**	28/07/2026	**https://github.com/Vaibhave-N/FDS-PRACTICAL.git**
2	**WEEK - 2**	04/08/2026	
3	**WEEK - 3**	25/08/2026	
4	**WEEK - 4**	29/09/2026	
5	**WEEK - 5**	06/10/2026	

FDS OBSERVATION

VAIBHAVE N
250701802

WEEK 1

1. Create a DataFrame from a dictionary
Q: Create a pandas DataFrame using the following dictionary:

```
import pandas as pd

data = {
   'Name': ['Alice', 'Bob', 'Charlie'],
   'Age': [25, 30, 35],
   'City': ['New York', 'Los Angeles', 'Chicago']
}

df = pd.DataFrame(data)
print(df)
```

OUTPUT:

```
     Name  Age        City
0   Alice   25     New York
1     Bob   30  Los Angeles
2  Charlie   35     Chicago
```

2.Read a CSV file
Q: Read a CSV file named data.csv into a DataFrame and print the first 5 rows.

```
import pandas as pd

df = pd.read_csv('data.csv')
print(df.head())
```

OUTPUT:

```
     Name  Age        City  Salary
0   Alice   25     New York   50000
1     Bob   30  Los Angeles   60000
2  Charlie   35     Chicago   70000
```

Filter rows based on a condition
Q: From a DataFrame df, print only the rows where Age is greater than 28.

```
print(df[df['Age'] > 28])
```

OUTPUT:

```
     Name  Age        City
1     Bob    30    Los Angeles
2  Charlie   35     Chicago
```

Add a new column
Q: Add a new column Salary with values [50000, 60000, 70000].

```
df['Salary'] = [50000, 60000, 70000]
print(df)

OUTPUT:
     Name  Age        City      Salary
0    Alice   25     New York     50000
1      Bob   30    Los Angeles   60000
2  Charlie   35      Chicago     70000
```

Group by a column and find the mean
Q: Given a DataFrame with columns Department and Salary, find the average salary per department.

```
avg_salary = df.groupby('Department')['Salary'].mean()
print(avg_salary)

OUTPUT:
Department
Finance    65000.0
HR         52500.0
IT         75000.0
Name: Salary, dtype: float64
```

Replace values in a column
Q: Replace all occurrences of 'New York' in City with 'NYC'.

```
df['City'] = df['City'].replace('New York', 'NYC')
print(df)

OUTPUT:
     Name  Age        City
0    Alice   25         NYC
1      Bob   30  Los Angeles
2  Charlie   35      Chicago
```

Drop a column
Q: Drop the Age column from the DataFrame.

```
df = df.drop(columns='Age')
print(df)

OUTPUT:
```

```
    Name        City  Salary
0   Alice        NYC   50000
1     Bob  Los Angeles   60000
2  Charlie     Chicago   70000
```

Sort the DataFrame by a column
Q: Sort the DataFrame by Salary in descending order.

```
df = df.sort_values(by='Salary', ascending=False)
print(df)
```

OUTPUT:

```
    Name        City     Salary
2  Charlie     Chicago   70000
1     Bob  Los Angeles   60000
0   Alice        NYC      50000
```

Find missing values
Q: Check for missing values in the DataFrame.

```
print(df.isnull().sum())
```

OUTPUT:

```
Name     0
City     0
Salary   0
dtype: int64
```

Save DataFrame to a CSV file
Q: Save the DataFrame to a CSV file named output.csv.

```
df.to_csv('output.csv', index=False)
```

OUTPUT:

```
Name,City,Salary

Charlie,Chicago,70000

Bob,Los Angeles,60000

Alice,NYC,50000
```

WEEK 2:

1. Create a DataFrame from the following data:

data = {

'Name': ['Alice', 'Bob', 'Charlie', 'David'],

'Age': [24, 27, 22, 32],

'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']

}

Write code to:

• a) Display the first two rows

• b) Print the column names

• c) Show the shape of the DataFrame

• d) Display the summary info of the DataFrame

```
import pandas as pd


data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [24, 27, 22, 32],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
}


df = pd.DataFrame(data)
```

```
# a) First two rows
print(df.head(2))

# b) Column names
print(df.columns)

# c) Shape (rows, columns)
print(df.shape)

# d) Summary info
df.info()
```

OUTPUT:

a) df.head(2)

```
    Name  Age         City
0  Alice   24     New York
1    Bob   27  Los Angeles
```

b) df.columns

```
    Name  Age         City
0  Alice   24     New York
1    Bob   27  Los Angeles
```

c) df.shape

```
(4, 3)
```

d) df.info()

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 4 entries, 0 to 3

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	Name	4 non-null	object
1	Age	4 non-null	int64
2	City	4 non-null	object

dtypes: int64(1), object(2)

memory usage: 228.0+ bytes

Part B: Selection & Indexing

2. Using the DataFrame above, write code to:

- a) Select only the Name column
- b) Select both Name and City columns
- c) Select the second row using .iloc
- d) Select the row where Name is 'Charlie' using .loc

```
# a) Select only the Name column
print(df['Name'])

# b) Select both Name and City columns
print(df[['Name', 'City']])
```

```
# c) Select the second row using .iloc

print(df.iloc[1])



# d) Select the row where Name is 'Charlie' using .loc

print(df.loc[df['Name'] == 'Charlie'])
```

OUTPUT:

a) df['Name']

```
0      Alice
1        Bob
2    Charlie
3      David
Name: Name, dtype: object
```

b) df[['Name', 'City']]

```
      Name         City
0    Alice     New York
1      Bob  Los Angeles
2  Charlie      Chicago
3    David      Houston
```

c) df.iloc[1]

```
Name              Bob
Age                27
City      Los Angeles
Name: 1, dtype: object
```

e) df.loc[df['Name'] == 'Charlie']

```
      Name  Age     City
2  Charlie   22  Chicago
```

f)
3. Filter the DataFrame to show:
• a) People older than 25
• b) People living in 'Chicago' or 'Houston'
• c) People whose age is between 23 and 30

```
# a) People older than 25
print(df[df['Age'] > 25])

# b) People living in 'Chicago' or 'Houston'
print(df[df['City'].isin(['Chicago', 'Houston'])])

# c) People whose age is between 23 and 30
print(df[df['Age'].between(23, 30)])
```

OUTPUT:
a) df[df['Age'] > 25]

```
    Name  Age         City
1    Bob   27  Los Angeles
3  David   32      Houston
```

b) df[df['City'].isin(['Chicago', 'Houston'])]

```
      Name  Age     City
2  Charlie   22  Chicago
3    David   32  Houston
```

c) df[df['Age'].between(23, 30)]

```
    Name  Age         City

0  Alice   24     New York

1    Bob   27  Los Angeles
```

4. Modify the DataFrame:

• a) Add a new column Score with values [85, 90, 88, 95]

• b) Change Bob’ s age to 28

• c) Remove the City column

• d) Drop the row of David

```
# a) Add a new column Score
df['Score'] = [85, 90, 88, 95]
print(df)

# b) Change Bob's age to 28
df.loc[df['Name'] == 'Bob', 'Age'] = 28
print(df)

# c) Remove the City column
df = df.drop(columns='City')
print(df)

# d) Drop the row of David
df = df.drop(df[df['Name'] == 'David'].index)
print(df)
```

OUTPUT:

a) After adding Score

	Name	Age	City	Score

0	Alice	24	New York	85
1	Bob	27	Los Angeles	90
2	Charlie	22	Chicago	88
3	David	32	Houston	95

b) After changing Bob’ s age

	Name	Age	City	Score
0	Alice	24	New York	85
1	Bob	28	Los Angeles	90
2	Charlie	22	Chicago	88
3	David	32	Houston	95

c) After removing City

	Name	Age	Score
0	Alice	24	85
1	Bob	28	90
2	Charlie	22	88
3	David	32	95

d) After dropping David’ s row

	Name	Age	Score
0	Alice	24	85
1	Bob	28	90
2	Charlie	22	88

5. Create a new DataFrame:

data = {

'Department': ['HR', 'IT', 'HR', 'IT'],

'Salary': [30000, 50000, 35000, 55000],

'Experience': [2, 5, 3, 6]

}

Write code to:

- a) Group by Department and find average Salary
- b) Find maximum Experience in each Department
- c) Calculate total Salary paid

```
import pandas as pd


data = {
    'Department': ['HR', 'IT', 'HR', 'IT'],
    'Salary': [30000, 50000, 35000, 55000],
    'Experience': [2, 5, 3, 6]
}


df2 = pd.DataFrame(data)


# a) Average Salary per Department
print(df2.groupby('Department')['Salary'].mean())
```

```
# b) Maximum Experience in each Department
print(df2.groupby('Department')['Experience'].max())

# c) Total Salary paid
print(df2['Salary'].sum())
```

OUTPUT:

a) Average Salary per Department

```
Department
HR    32500.0
IT    52500.0
Name: Salary, dtype: float64
```

b) Maximum Experience per Department

```
Department
HR    3
IT    6
Name: Experience, dtype: int64
```

c) Total Salary paid

```
170000
```

6. Given a DataFrame with missing values:

data = {

'Student': ['John', 'Emma', 'Sam', 'Olivia'],

'Marks': [80, None, 75, 90]

}

Write code to:

- a) Fill missing marks with 0
- b) Drop rows with missing values
- c) Sort the DataFrame by Marks in descending order

```
import pandas as pd


data = {
    'Student': ['John', 'Emma', 'Sam', 'Olivia'],
    'Marks': [80, None, 75, 90]
}


df3 = pd.DataFrame(data)


# a) Fill missing marks with 0
print(df3.fillna(0))


# b) Drop rows with missing values
print(df3.dropna())
```

```
# c) Sort by Marks in descending order
print(df3.sort_values(by='Marks', ascending=False))
```

OUTPUT:

a) df3.fillna(0)

```
  Student  Marks
0    John   80.0
1    Emma    0.0
2     Sam   75.0
3  Olivia   90.0
```

b) df3.dropna()

```
  Student  Marks
0    John   80.0
2     Sam   75.0
3  Olivia   90.0
```

c) df3.sort_values(by='Marks', ascending=False)

```
  Student  Marks
3  Olivia   90.0
0    John   80.0
2     Sam   75.0
1    Emma    NaN
```

7. For any DataFrame:

• a) Save it as a CSV file named students.csv

• b) Load a CSV file named employees.csv

• c) Set Name as the index

• d) Reset the index

```
import pandas as pd


df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [24, 27, 22, 32],
    'City': ['New York', 'Los Angeles', 'Chicago', 'Houston']
})


# a) Save as students.csv
df.to_csv('students.csv', index=False)


# b) Load employees.csv
emp = pd.read_csv('employees.csv')
print(emp.head())


# c) Set Name as the index
df = df.set_index('Name')
print(df)
```

```
# d) Reset the index
df = df.reset_index()
print(df)
```

OUTPUT:

a) df.to_csv('students.csv', index=False)

```
Name,Age,City
Alice,24,New York
Bob,27,Los Angeles
Charlie,22,Chicago
David,32,Houston
```

b) pd.read_csv('employees.csv')

	EmpID	Name	Department	Salary
0	101	Ravi	HR	30000
1	102	Priya	IT	50000
2	103	Kiran	HR	35000

c) df.reset_index()

	Name	Age	City
0	Alice	24	New York
1	Bob	27	Los Angeles
2	Charlie	22	Chicago

3 David 32 Houston

8) You are given the following DataFrame:

```
import pandas as pd

data = {

'Product': ['Laptop', 'Tablet', 'Smartphone', 'Monitor', 'Keyboard'],

'Price': [70000, 30000, 25000, 15000, 2000],

'Stock': [10, 25, 50, 15, 100]

}

df = pd.DataFrame(data)
```

Task:

Write a line of code using .loc[] to display the details of the first and

third products in the DataFrame.

Expected Output:

Product Price Stock

0 Laptop 70000 10

2 Smartphone 25000 50

```
import pandas as pd


data = {

    'Product': ['Laptop', 'Tablet', 'Smartphone', 'Monitor', 'Keyboard'],

    'Price': [70000, 30000, 25000, 15000, 2000],
```

```
    'Stock': [10, 25, 50, 15, 100]
}
df = pd.DataFrame(data)

print(df.loc[[0, 2]])
```

OUTPUT:

```
       Product  Price  Stock
0       Laptop  70000     10
2   Smartphone  25000     50
```

You are given the following data:

```
import pandas as pd
data = {
'Subject': ['Math', 'Science', 'English'],
'Marks': [88, 92, 85]
}
```

Task:
Create a DataFrame from the above data and assign custom row labels: 'Student1', 'Student2', and 'Student3' using the index argument.
Then, using .loc[], print the marks obtained by 'Student2'.

```
import pandas as pd

data = {
   'Subject': ['Math', 'Science', 'English'],
   'Marks': [88, 92, 85]
}

df = pd.DataFrame(data, index=['Student1', 'Student2', 'Student3'])
print(df)
```

```
print(df.loc['Student2', 'Marks'])
```

OUTPUT:

```
            Subject  Marks
Student1    Math     88
Student2   Science    92
Student3   English    85
```

WEEK 3

Question 1 – Find Null Values
Create a Pandas DataFrame containing student details such as Name, Age, Marks, and City. Some values should be
missing.
Write a Python program to:
1. Display the dataset.
2. Find the number of null values in each column.
3. Display the rows containing null values.
Answer / Program

```
import pandas as pd
data = {
'Name': ['Arun', 'Binu', 'Charan', 'Divya'],
'Age': [20, None, 21, None],
'Marks': [85, 78, None, 92],
'City': ['Chennai', 'Kochi', None, 'Madurai']
}
df = pd.DataFrame(data)
print("Dataset:")
print(df)
print("\nNumber of null values:")
print(df.isnull().sum())
print("\nRows containing null values:")
print(df[df.isnull().any(axis=1)])
```

Output:

```
Dataset:
    Name   Age  Marks     City
0    Arun  20.0   85.0  Chennai
1    Binu   NaN   78.0    Kochi
2  Charan  21.0    NaN     None
3   Divya   NaN   92.0  Madurai

Number of null values:
Name     0
Age      2
Marks    1
City     1
dtype: int64

Rows containing null values:
    Name   Age  Marks     City
1    Binu   NaN   78.0    Kochi
2  Charan  21.0    NaN     None
3   Divya   NaN   92.0  Madurai
```

Question 2 – Fill Missing Values
Consider the student dataset: Arun (20, 85), Binu (NaN, 78), Charan (21, NaN), Divya (NaN, 92), Esha (20, 88).

Write a Python program to find missing values and replace missing Age and Marks values with their respective mean.

Answer / Program

```
import pandas as pd
data = {
'Name': ['Arun', 'Binu', 'Charan', 'Divya', 'Esha'],
'Age': [20, None, 21, None, 20],
'Marks': [85, 78, None, 92, 88]
}
df = pd.DataFrame(data)
print("Missing Values:")
print(df.isnull().sum())
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())
print("\nAfter Filling Missing Values:")
print(df)
```

Output:

```
Missing Values:
Name    0
Age     2
Marks   1
dtype: int64

After Filling Missing Values:
    Name       Age        Marks
0   Arun    20.000000   85.0
1   Binu    20.333333   78.0
2  Charan  21.000000   85.75
3   Divya   20.333333   92.0
4   Esha    20.000000   88.0
```

Question 3 – Remove Missing Records

Create an employee DataFrame containing Emp_ID, Name, Department, and Salary, with some missing values. Identify
missing values, remove rows containing missing values, and display the cleaned dataset.

Answer / Program

```
import pandas as pd
data = {
'Emp_ID': [101, 102, 103, 104],
'Name': ['Arun', 'Binu', 'Charan', 'Divya'],
'Department': ['CSE', None, 'ECE', 'CSE'],
'Salary': [40000, 45000, None, 50000]
}
df = pd.DataFrame(data)
print("Missing Values:")
print(df.isnull().sum())
df_clean = df.dropna()
print("\nAfter Removing Missing Records:")
print(df_clean)
```

Output:
Missing Values:
Emp_ID 0
Name 0
Department 1
Salary 1
dtype: int64

After Removing Missing Records:
Emp_ID Name Department Salary
0 101 Arun CSE 40000.0
3 104 Divya CSE 50000.0

Question 4 – Detect Duplicate Records
Create a customer dataset with Customer_ID, Name, and City, including a duplicate record. Identify duplicate records,
count them, and remove them.
Answer / Program

```
import pandas as pd
data = {
'Customer_ID': [101, 102, 103, 102, 104],
'Name': ['Arun', 'Binu', 'Divya', 'Binu', 'Esha'],
'City': ['Chennai', 'Kochi', 'Madurai', 'Kochi', 'Chennai']
}
df = pd.DataFrame(data)
print("Duplicate Records:")
print(df[df.duplicated()])
print("\nNumber of Duplicate Records:")
print(df.duplicated().sum())
df = df.drop_duplicates()
print("\nAfter Removing Duplicates:")
print(df)
```

Output:
Duplicate Records:
Customer_ID Name City
3 102 Binu Kochi

Number of Duplicate Records:
1

After Removing Duplicates:
Customer_ID Name City
0 101 Arun Chennai
1 102 Binu Kochi
2 103 Divya Madurai
4 104 Esha Chennai

Question 5 – Forward Fill and Backward Fill
Create a temperature dataset for Monday to Friday with missing values on Tuesday and Thursday. Identify missing
values and fill them using forward fill (ffill) and backward fill (bfill).
Answer / Program

```
import pandas as pd
data = {
```

```
'Day': ['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday'],
'Temperature': [30, None, 32, None, 35]
}
df = pd.DataFrame(data)
df_ffill = df.copy()
df_ffill['Temperature'] = df_ffill['Temperature'].ffill()
print("After Forward Fill:")
print(df_ffill)
df_bfill = df.copy()
df_bfill['Temperature'] = df_bfill['Temperature'].bfill()
print("\nAfter Backward Fill:")
print(df_bfill)
```

Output:

```
After Forward Fill:
        Day  Temperature
0    Monday         30.0
1   Tuesday         30.0
2 Wednesday         32.0
3  Thursday         32.0
4    Friday         35.0

After Backward Fill:
        Day  Temperature
0    Monday         30.0
1   Tuesday         32.0
2 Wednesday         32.0
3  Thursday         35.0
4    Friday         35.0
```

Question 6 – Find Incorrect Data Format

Create an employee dataset containing Name, Age, and Joining_Date with dates written in different formats. Convert Joining_Date to a proper Pandas datetime format and identify invalid date values.

Answer / Program

```
import pandas as pd
data = {
'Name': ['Arun', 'Binu', 'Charan', 'Divya'],
'Age': [25, 26, 24, 27],
'Joining_Date': ['2024-01-10', '15-02-2024',
'2024/03/20', '2024-04-25']
}
df = pd.DataFrame(data)
df['Joining_Date'] = pd.to_datetime(
df['Joining_Date'], dayfirst=True, errors='coerce'
)
print("After Converting to Date Format:")
print(df)
print("\nInvalid Dates:")
print(df[df['Joining_Date'].isnull()])
```

Output:

```
After Converting to Date Format:
    Name  Age Joining_Date
```

```
0   Arun   25   2024-01-10
1   Binu   26         NaT
2  Charan   24   2024-03-20
3   Divya   27   2024-04-25

Invalid Dates:
  Name  Age   Joining_Date
1  Binu   26           NaT
```

Question 7 – Detect and Correct Inconsistent Data
Create employee data containing department values such as CSE, cse, Computer Science, ECE, and ece. Identify
inconsistent values, convert department names to uppercase, and replace COMPUTER SCIENCE with CSE.
Answer / Program

```
import pandas as pd
data = {
'Name': ['Arun', 'Binu', 'Charan', 'Divya', 'Esha'],
'Department': ['CSE', 'cse', 'Computer Science', 'ECE', 'ece']
}
df = pd.DataFrame(data)
df['Department'] = df['Department'].str.upper()
df['Department'] = df['Department'].replace(
'COMPUTER SCIENCE', 'CSE'
)
print("After Data Cleaning:")
print(df)
```

OUTPUT:

```
After Data Cleaning:
    Name Department
0   Arun       CSE
1   Binu       CSE
2  Charan       CSE
3   Divya       ECE
4   Esha       ECE
```

Question 8 – Detect Incorrect Age Values
Create a student dataset containing valid and invalid ages such as -5 and 150. Identify invalid ages, replace them with
NaN, and fill them using the mean of valid ages.
Answer / Program

```
import pandas as pd
import numpy as np
data = {
'Name': ['Arun', 'Binu', 'Charan', 'Divya', 'Esha'],
'Age': [20, 21, -5, 150, 22],
'Marks': [85, 78, 90, 88, 95]
}
df = pd.DataFrame(data)
invalid = (df['Age'] < 0) | (df['Age'] > 100)
```

```
print("Invalid Age Records:")
print(df[invalid])
df.loc[invalid, 'Age'] = np.nan
df['Age'] = df['Age'].fillna(df['Age'].mean())
print("\nFinal Dataset:")
print(df)
```

OUTPUT:

```
Invalid Age Records:
    Name  Age  Marks
2  Charan   -5     90
3   Divya  150     88

Final Dataset:
    Name       Age  Marks
0   Arun  20.000000     85
1   Binu  21.000000     78
2  Charan  21.000000     90
3   Divya  21.000000     88
4   Esha  22.000000     95
```

Question 9 – Complete Data Cleaning
Create a customer dataset with ID, Name, Age, and Email containing missing values and a duplicate record. Find
missing values, remove duplicates, fill missing Age with the mean, fill missing Email with “Not Provided”, and display the
final dataset.
Answer / Program

```
import pandas as pd
data = {
'ID': [101, 102, 103, 102, 104],
'Name': ['Arun', 'Binu', 'Charan', 'Binu', 'Divya'],
'Age': [25, None, 30, None, 28],
'Email': ['arun@gmail.com', 'binu@gmail.com', None,
'binu@gmail.com', 'divya@gmail.com']
}
df = pd.DataFrame(data)
print("Missing Values:")
print(df.isnull().sum())
df = df.drop_duplicates()
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Email'] = df['Email'].fillna('Not Provided')
print("\nFinal Cleaned Dataset:")
print(df)
```

OUTPUT:

```
Missing Values:
ID      0
Name    0
Age     2
Email   1
```

dtype: int64

Final Cleaned Dataset:

	ID	Name	Age	Email
0	101	Arun	25.000000	arun@gmail.com
1	102	Binu	27.666667	binu@gmail.com
2	103	Charan	30.000000	Not Provided
4	104	Divya	28.000000	divya@gmail.com

Question 10 – Complete Data Preprocessing

Create a student dataset containing missing values, a duplicate record, an invalid age, inconsistent department names,
and a missing mark. Perform all relevant cleaning operations: find missing values, remove duplicates, identify and
replace invalid age with NaN, fill missing Age and Marks with means, convert Department to uppercase, replace
COMPUTER SCIENCE with CSE, and display the final dataset.

Answer / Program

```
import pandas as pd
import numpy as np
data = {
'ID': [101, 102, 103, 102, 104, 105],
'Name': ['Arun', 'Binu', 'Charan', 'Binu', 'Divya', 'Esha'],
'Age': [20, None, 21, None, 200, 22],
'Marks': [85, 78, None, 78, 92, 88],
'Department': ['CSE', 'cse', 'CSE', 'cse',
'Computer Science', 'ECE']
}
df = pd.DataFrame(data)
print("Missing Values:")
print(df.isnull().sum())
df = df.drop_duplicates()
invalid_age = (df['Age'] < 0) | (df['Age'] > 100)
print("\nInvalid Age Records:")
print(df[invalid_age])
df.loc[invalid_age, 'Age'] = np.nan
df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Marks'] = df['Marks'].fillna(df['Marks'].mean())
df['Department'] = df['Department'].str.upper()
df['Department'] = df['Department'].replace(
'COMPUTER SCIENCE', 'CSE'
)
print("\nFinal Cleaned Dataset:")
print(df)
```

Output:

Missing Values:

ID	0
Name	0
Age	2
Marks	1
Department	0

dtype: int64

Invalid Age Records:

	ID	Name	Age	Marks	Department
4	104	Divya	200.0	92.0	Computer Science

Final Cleaned Dataset:

	ID	Name	Age	Marks	Department
0	101	Arun	20.0	85.00	CSE
1	102	Binu	21.0	78.00	CSE
2	103	Charan	21.0	85.75	CSE
4	104	Divya	21.0	92.00	CSE
5	105	Esha	22.0	88.00	ECE

Week 4

Q1. Analysis of Data Science Job Postings

A recruitment analytics company has collected the number of Data Science job postings recorded each year over a 10-year period. Analyze the dataset using Pandas and visualize the trend using Matplotlib/Seaborn.

Year Data_Science_Job_Postings
2015 12500
2016 14800
2017 17600
2018 21500
2019 26800
2020 24100
2021 35200
2022 42800
2023 48700
2024 53100

Write a Python program using Pandas and Matplotlib/Seaborn to perform the following:

1. Create a Pandas DataFrame using the given dataset.
2. Display the first five and last five records.
3. Display the basic statistical summary of the dataset.
4. Identify the year with the highest number of job postings.
5. Identify the year with the lowest number of job postings.
6. Calculate the average number of job postings over the 10-year period.
7. Calculate the year-to-year change in the number of job postings.
8. Identify the year that experienced the largest increase in job postings.
9. Plot a line chart showing the trend of Data Science job postings from 2015 to 2024.
10. Add an appropriate title, X-axis label, Y-axis label, markers, and grid to the chart.
11. Using the visualization, write 2–3 observations about the trend in Data Science job postings.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Create the DataFrame
data = {
   'Year': [2015, 2016, 2017, 2018, 2019, 2020, 2021, 2022, 2023, 2024],
   'Data_Science_Job_Postings': [12500, 14800, 17600, 21500, 26800,
                                 24100, 35200, 42800, 48700, 53100]
}
df = pd.DataFrame(data)

# 2. First five and last five records
print("First 5 Records:")
print(df.head())
print("\nLast 5 Records:")
print(df.tail())

# 3. Statistical summary
print("\nStatistical Summary:")
print(df['Data_Science_Job_Postings'].describe())

# 4. Year with the highest postings
max_row = df.loc[df['Data_Science_Job_Postings'].idxmax()]
print("\nHighest postings:", int(max_row['Year']), "->", int(max_row['Data_Science_Job_Postings']))
```

```
# 5. Year with the lowest postings
min_row = df.loc[df['Data_Science_Job_Postings'].idxmin()]
print("Lowest postings:", int(min_row['Year']), "->", int(min_row['Data_Science_Job_Postings']))

# 6. Average postings over 10 years
print("\nAverage postings:", df['Data_Science_Job_Postings'].mean())

# 7. Year-to-year change
df['Change'] = df['Data_Science_Job_Postings'].diff()
print("\nYear-to-Year Change:")
print(df[['Year', 'Change']])

# 8. Year with the largest increase
max_inc = df.loc[df['Change'].idxmax()]
print("\nLargest increase:", int(max_inc['Year']), "->", int(max_inc['Change']))

# 9 & 10. Line chart with title, labels, markers and grid
sns.set_style("whitegrid")
plt.figure(figsize=(10, 6))
plt.plot(df['Year'], df['Data_Science_Job_Postings'],
       marker='o', linewidth=2, color='royalblue')
plt.title('Trend of Data Science Job Postings (2015-2024)')
plt.xlabel('Year')
plt.ylabel('Number of Job Postings')
plt.xticks(df['Year'])
plt.grid(True)
plt.show()
```

Output:

First 5 Records:

	Year	Data_Science_Job_Postings
0	2015	12500
1	2016	14800
2	2017	17600
3	2018	21500
4	2019	26800

Last 5 Records:

	Year	Data_Science_Job_Postings
5	2020	24100
6	2021	35200
7	2022	42800
8	2023	48700
9	2024	53100

Statistical Summary:

```
count       10.000000
mean     29710.000000
std      14459.479800
min      12500.000000
25%      18575.000000
50%      25450.000000
75%      40900.000000
max      53100.000000
Name: Data_Science_Job_Postings, dtype: float64
```

```
Highest postings: 2024 -> 53100
Lowest postings: 2015 -> 12500

Average postings: 29710.0

Year-to-Year Change:
   Year   Change
0  2015      NaN
1  2016   2300.0
2  2017   2800.0
3  2018   3900.0
4  2019   5300.0
5  2020  -2700.0
6  2021  11100.0
7  2022   7600.0
8  2023   5900.0
9  2024   4400.0

Largest increase: 2021 -> 11100
```

Q2. Analysis of Data Science Job Roles

A technology recruitment company has collected information about job postings for different Data Science-related roles. Analyze the distribution of these roles using Pandas and visualize the results using Matplotlib/Seaborn.

Job_Role Number_of_Postings
Data Analyst 3200
Data Scientist 2800
Data Engineer 3500
Machine Learning Engineer 2400
Business Intelligence Analyst 1900
AI Engineer 2100
Machine Learning Scientist 1200
Data Architect 900
Statistician 700
Research Scientist 600

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Create the DataFrame
data = {
   'Job_Role': ['Data Analyst', 'Data Scientist', 'Data Engineer',
            'Machine Learning Engineer', 'Business Intelligence Analyst',
            'AI Engineer', 'Machine Learning Scientist',
            'Data Architect', 'Statistician', 'Research Scientist'],
   'Number_of_Postings': [3200, 2800, 3500, 2400, 1900,
                     2100, 1200, 900, 700, 600]
}
df = pd.DataFrame(data)
```

```
print("Dataset:")
print(df)

# 2. Statistical summary
print("\nStatistical Summary:")
print(df['Number_of_Postings'].describe())

# 3. Role with the most and fewest postings
top = df.loc[df['Number_of_Postings'].idxmax()]
low = df.loc[df['Number_of_Postings'].idxmin()]
print("\nMost postings:", top['Job_Role'], "->", top['Number_of_Postings'])
print("Fewest postings:", low['Job_Role'], "->", low['Number_of_Postings'])

# 4. Total and average postings
total = df['Number_of_Postings'].sum()
print("\nTotal postings:", total)
print("Average postings per role:", df['Number_of_Postings'].mean())

# 5. Percentage share, sorted from highest to lowest
df['Percentage'] = (df['Number_of_Postings'] / total * 100).round(2)
df_sorted = df.sort_values(by='Number_of_Postings', ascending=False).reset_index(drop=True)
print("\nSorted with Percentage Share:")
print(df_sorted)

# 6. Roles above the average
avg = df['Number_of_Postings'].mean()
print("\nRoles above average:")
print(df[df['Number_of_Postings'] > avg]['Job_Role'].tolist())

# 7. Horizontal bar chart
sns.set_style("whitegrid")
plt.figure(figsize=(10, 6))
sns.barplot(x='Number_of_Postings', y='Job_Role', data=df_sorted, palette='viridis')
plt.title('Number of Job Postings by Data Science Role')
plt.xlabel('Number of Postings')
plt.ylabel('Job Role')
plt.tight_layout()
plt.show()

# 8. Pie chart of share
plt.figure(figsize=(8, 8))
plt.pie(df_sorted['Number_of_Postings'], labels=df_sorted['Job_Role'],
      autopct='%1.1f%%', startangle=90)
plt.title('Share of Job Postings by Role')
plt.show()
```

Output:

Dataset:

	Job_Role	Number_of_Postings
0	Data Analyst	3200
1	Data Scientist	2800
2	Data Engineer	3500
3	Machine Learning Engineer	2400
4	Business Intelligence Analyst	1900
5	AI Engineer	2100

```
6    Machine Learning Scientist             1200
7              Data Architect              900
8               Statistician              700
9          Research Scientist              600

Statistical Summary:
count      10.000000
mean     1930.000000
std      1052.035...
min       600.000000
25%       975.000000
50%      2000.000000
75%      2700.000000
max      3500.000000
Name: Number_of_Postings, dtype: float64

Most postings: Data Engineer -> 3500
Fewest postings: Research Scientist -> 600

Total postings: 19300
Average postings per role: 1930.0

Sorted with Percentage Share:
                  Job_Role            Number_of_Postings     Percentage
0              Data Engineer              3500                  18.13
1               Data Analyst              3200                   16.58
2             Data Scientist               2800                  14.51
3     Machine Learning Engineer      2400                  12.44
4                AI Engineer                2100                   10.88
5  Business Intelligence Analyst         1900                    9.84
6    Machine Learning Scientist          1200                   6.22
7              Data Architect             900                    4.66
8               Statistician              700                    3.63
9          Research Scientist             600                    3.11

Roles above average:
['Data Analyst', 'Data Scientist', 'Data Engineer', 'Machine Learning Engineer', 'AI Engineer']
```

Q3. Classification of Structured, Semi-Structured and Unstructured Data

A Data Science application receives data in different formats. Write a Python program to create and analyze examples of structured, semi-structured, and unstructured data and differentiate them based on their organization and schema.
Create the following datasets:
1. Structured Data: Create a Pandas DataFrame containing student details such as ID, Name, Age, and Department.
2. Semi-Structured Data: Create a nested Python dictionary representing an employee/student record, containing basic details and an Attributes field with additional information.
3. Unstructured Data: Create a text variable containing a paragraph of free-form information such as customer feedback, student feedback, or a review.
Perform the following operations:
• Display each dataset separately.
• Display the data type of each dataset using Python.
• Access and display selected values from the structured and semi-structured data.
• Display the nested attributes from the semi-structured data.

• Display the complete unstructured text.
• Explain why each dataset belongs to the corresponding category.
• Create a comparison table showing the differences between the three types of data based on:
o Organization
o Schema
o Data format
o Ease of processing
o Example applications

```
import pandas as pd

# 1. Structured data: a table with fixed columns
structured = pd.DataFrame({
   'ID': [101, 102, 103, 104],
   'Name': ['Arun', 'Binu', 'Charan', 'Divya'],
   'Age': [20, 21, 22, 20],
   'Department': ['CSE', 'ECE', 'CSE', 'IT']
})

# 2. Semi-structured data: nested dictionary (JSON-like)
semi_structured = {
   'ID': 201,
   'Name': 'Esha',
   'Age': 21,
   'Department': 'CSE',
   'Attributes': {
      'Skills': ['Python', 'SQL', 'Machine Learning'],
      'Email': 'esha@gmail.com',
      'Certifications': {
         'Course': 'Data Science',
         'Year': 2024
      }
   }
}

# 3. Unstructured data: free-form text
unstructured = ("The course was very helpful and the faculty explained every "
            "concept clearly. The practical sessions were interesting, but "
            "the lab timings were too short. Overall, I enjoyed learning "
            "Data Science and would recommend it to other students.")

# Display each dataset
print("STRUCTURED DATA:")
print(structured)

print("\nSEMI-STRUCTURED DATA:")
print(semi_structured)

print("\nUNSTRUCTURED DATA:")
print(unstructured)

# Data types
print("\nData Types:")
print("Structured     :", type(structured))
print("Semi-structured :", type(semi_structured))
```

```
print("Unstructured    :", type(unstructured))

# Access selected values
print("\nSelected values from structured data:")
print("Name column:")
print(structured['Name'])
print("Details of ID 102:")
print(structured[structured['ID'] == 102])

print("\nSelected values from semi-structured data:")
print("Name:", semi_structured['Name'])
print("Department:", semi_structured['Department'])

# Nested attributes
print("\nNested Attributes:")
print("Skills:", semi_structured['Attributes']['Skills'])
print("Email:", semi_structured['Attributes']['Email'])
print("Certification:", semi_structured['Attributes']['Certifications'])

# Complete unstructured text
print("\nComplete Unstructured Text:")
print(unstructured)
```

Output:

```
STRUCTURED DATA:
   ID    Name  Age Department
0  101    Arun   20       CSE
1  102    Binu   21       ECE
2  103  Charan   22       CSE
3  104   Divya   20        IT

SEMI-STRUCTURED DATA:
{'ID': 201, 'Name': 'Esha', 'Age': 21, 'Department': 'CSE', 'Attributes': {'Skills': ['Python', 'SQL', 'Machine
Learning'], 'Email': 'esha@gmail.com', 'Certifications': {'Course': 'Data Science', 'Year': 2024}}}

UNSTRUCTURED DATA:
The course was very helpful and the faculty explained every concept clearly. The practical sessions
were interesting, but the lab timings were too short. Overall, I enjoyed learning Data Science and
would recommend it to other students.

Data Types:
Structured     : <class 'pandas.core.frame.DataFrame'>
Semi-structured : <class 'dict'>
Unstructured    : <class 'str'>

Selected values from structured data:
Name column:
0     Arun
1     Binu
2   Charan
3    Divya
Name: Name, dtype: object
Details of ID 102:
   ID  Name  Age Department
1  102  Binu   21       ECE
```

Selected values from semi-structured data:
Name: Esha
Department: CSE

Nested Attributes:
Skills: ['Python', 'SQL', 'Machine Learning']
Email: esha@gmail.com
Certification: {'Course': 'Data Science', 'Year': 2024}

Complete Unstructured Text:
The course was very helpful and the faculty explained every concept clearly. The practical sessions were interesting, but the lab timings were too short. Overall, I enjoyed learning Data Science and would recommend it to other students.

WEEK 5

Q) Write a Python program to count the frequency of occurrence of each word in a given body of text. Display each word along with its corresponding frequency.

```
text = input("Enter the text: ")
words = text.lower().split()
frequency = {}
for word in words:
if word in frequency:
frequency[word] += 1
else:
frequency[word] = 1
print("\nWord Frequency:")
for word, count in frequency.items():
print(word, ":", count)
```

```
text = input("Enter the text: ")
words = text.lower().split()
frequency = {}
for word in words:
    if word in frequency:
        frequency[word] += 1
    else:
        frequency[word] = 1
print("\nWord Frequency:")
for word, count in frequency.items():
    print(word, ":", count)
```

OUTPUT:

```
Word Frequency:
data : 2
science : 2
is : 2
fun : 1
and : 1
useful : 1
```

Q) Write a Python program to collect a dataset from a CSV file and perform initial data exploration using Pandas. Display the first five and last five records, dataset dimensions, column names, data types, summary statistics, and check for missing values.

```
import pandas as pd

# Load the dataset from a CSV file
df = pd.read_csv('data.csv')

# 1. First five and last five records
print("First 5 Records:")
print(df.head())
print("\nLast 5 Records:")
print(df.tail())
```

```
# 2. Dataset dimensions
print("\nDataset Dimensions (rows, columns):", df.shape)
print("Number of Rows   :", df.shape[0])
print("Number of Columns:", df.shape[1])

# 3. Column names
print("\nColumn Names:")
print(list(df.columns))

# 4. Data types
print("\nData Types:")
print(df.dtypes)

# 5. Summary statistics
print("\nSummary Statistics:")
print(df.describe())

# 6. Missing values
print("\nMissing Values in Each Column:")
print(df.isnull().sum())
```

OUTPUT:

```
First 5 Records:
    Name  Age    City  Marks
0   Arun   20  Chennai   85.0
1   Binu   21   Kochi   78.0
2  Charan   22  Madurai    NaN
3   Divya   20  Chennai   92.0
4   Esha   23   Kochi   88.0

Last 5 Records:
    Name  Age    City  Marks
2  Charan   22  Madurai    NaN
3   Divya   20  Chennai   92.0
4   Esha   23   Kochi   88.0
5   Farid   21  Madurai   75.0
6  George   22  Chennai   90.0

Dataset Dimensions (rows, columns): (7, 4)
Number of Rows   : 7
Number of Columns: 4

Column Names:
['Name', 'Age', 'City', 'Marks']

Data Types:
Name      object
Age        int64
City      object
Marks    float64
dtype: object

Summary Statistics:
             Age      Marks
```

```
count   7.000000   6.000000
mean   21.285714  84.666667
std     1.112697   6.947422
min    20.000000  75.000000
25%    20.500000  79.750000
50%    21.000000  86.500000
75%    22.000000  89.500000
max    23.000000  92.000000

Missing Values in Each Column:
Name     0
Age      0
City     0
Marks    1
dtype: int64
```

Q) Write a Python program to perform time series analysis of website traffic data. Load the dataset containing date-wise website visits, convert the date column into a datetime format, visualize the website traffic over time using a line chart, and identify the trend in website traffic.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Load the dataset
# With a CSV file, use: df = pd.read_csv('website_traffic.csv')
data = {
   'Date': ['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04', '2024-01-05',
            '2024-01-06', '2024-01-07', '2024-01-08', '2024-01-09', '2024-01-10'],
   'Visits': [1200, 1350, 1280, 1500, 1620, 1580, 1750, 1900, 1850, 2100]
}
df = pd.DataFrame(data)

# 2. Convert the Date column to datetime and use it as the index
df['Date'] = pd.to_datetime(df['Date'])
df = df.set_index('Date')

print("Dataset:")
print(df)
print("\nIndex type:", df.index.dtype)

# 3. Basic statistics
print("\nSummary Statistics:")
print(df['Visits'].describe())
print("\nHighest traffic:", df['Visits'].idxmax().date(), "->", df['Visits'].max())
print("Lowest traffic :", df['Visits'].idxmin().date(), "->", df['Visits'].min())

# 4. Trend: 3-day moving average and a linear trend line
df['Moving_Avg_3'] = df['Visits'].rolling(window=3).mean()

x = np.arange(len(df))
slope, intercept = np.polyfit(x, df['Visits'], 1)
df['Trend_Line'] = slope * x + intercept
print("\nTrend slope: {:.2f} visits per day".format(slope))
print("Trend direction:", "Upward" if slope > 0 else "Downward" if slope < 0 else "Flat")
```

```
# 5. Line chart
plt.figure(figsize=(10, 6))
plt.plot(df.index, df['Visits'], marker='o', label='Daily Visits')
plt.plot(df.index, df['Moving_Avg_3'], linestyle='--', label='3-Day Moving Average')
plt.plot(df.index, df['Trend_Line'], linestyle=':', color='red', label='Linear Trend')
plt.title('Website Traffic Over Time')
plt.xlabel('Date')
plt.ylabel('Number of Visits')
plt.xticks(rotation=45)
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.show()
```

OUTPUT:

Dataset:

Date	Visits
2024-01-01	1200
2024-01-02	1350
2024-01-03	1280
2024-01-04	1500
2024-01-05	1620
2024-01-06	1580
2024-01-07	1750
2024-01-08	1900
2024-01-09	1850
2024-01-10	2100

Index type: datetime64[ns]

Summary Statistics:

count	10.000000
mean	1613.000000
std	290.404...
min	1200.000000
25%	1387.500000
50%	1600.000000
75%	1825.000000
max	2100.000000

Name: Visits, dtype: float64

Highest traffic: 2024-01-10 -> 2100
Lowest traffic : 2024-01-01 -> 1200

Trend slope: 93.39 visits per day
Trend direction: Upward

Q) Write a Python program to perform random sampling on a given dataset. Select multiple random
samples of a specified size, calculate the mean of each sample, and visualize the sampling
distribution of the sample means using a histogram.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# Set random seed for reproducibility
np.random.seed(42)

# 1. Create or load a dataset (Example: Population of 100,000 values)
# Generating a skewed population dataset to observe the Central Limit Theorem
population_data = np.random.exponential(scale=10, size=100000)

# 2. Define sampling parameters
sample_size = 50  # Size of each sample (n)
num_samples = 1000  # Number of random samples to draw

# 3. Perform random sampling and calculate sample means
sample_means = []

for _ in range(num_samples):
    # Draw a random sample of specified size with replacement
    sample = np.random.choice(population_data, size=sample_size, replace=True)
    # Calculate and store the mean of the sample
    sample_means.append(np.mean(sample))

# 4. Plot the sampling distribution of the sample means
plt.figure(figsize=(10, 6))

# Histogram of sample means
plt.hist(
    sample_means,
    bins=30,
    edgecolor="black",
    alpha=0.7,
    color="skyblue",
    density=True,
)

# Overlay theoretical normal distribution curve
mean_of_means = np.mean(sample_means)
std_of_means = np.std(sample_means)
xmin, xmax = plt.xlim()
x = np.linspace(xmin, xmax, 100)
p = (1 / (std_of_means * np.sqrt(2 * np.pi))) * np.exp(
    -0.5 * ((x - mean_of_means) / std_of_means) ** 2
)
plt.plot(x, p, "k--", linewidth=2, label="Normal Curve Fit")
```

```
# Formatting the plot
plt.title(
   f"Sampling Distribution of Sample Means\n({num_samples} samples of size n={sample_size})",
   fontsize=14,
)
plt.xlabel("Sample Mean", fontsize=12)
plt.ylabel("Density", fontsize=12)
plt.axvline(
   mean_of_means,
   color="red",
   linestyle="dashed",
   linewidth=2,
   label=f"Mean of Means: {mean_of_means:.2f}",
)
plt.legend(fontsize=10)
plt.grid(axis="y", alpha=0.3)

# Show the plot
plt.tight_layout()
plt.show()
```

OUTPUT:

```
Population Mean: 10.02
Mean of Sample Means: 10.01
Standard Error of Mean (Theoretical): 1.41
Standard Deviation of Sample Means (Empirical): 1.40
```